Intro to Data Science - HW 6

Copyright Jeffrey Stanton 2025, Jeffrey Saltz, Christopher Dunham, and Jasmina Tacheva

```
# Enter your name here: Tim Soron
```

USE OF ARTIFICIAL INTELLIGENCE: While you are allowed to use generative AI as a learning tool for this assignment (e.g., to help with brainstorming, to see how to code a certain concept, to help identify flaws in reasoning, or to spot confusing or underdeveloped code segments), it must not be used to input homework questions, problems, or instructions directly, and you may not copy and paste any AI-generated content into your work. By submitting this assignment, you confirm that all the work is your own.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

Data visualization is important because many people can understand data better when it is presented graphically. As a data scientist, you will have to present complex data to decision makers in a way they can understand. Based on lecture and your experience with Excel and other tools, you know there are numerous data visualization types (e.g., line charts). How many of them can you name?

The most powerful tool for data visualization in R is called ggplot. Written by computer/data scientist Hadley Wickham, this graphics grammar tool builds visualizations in layers. This method provides immense flexibility, but takes a bit of practice to master.

Step 1: Make a copy of the data

A. Read the **Syracuse Code Violations** data from this URL: "https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv" into a dataframe called **violations_df**.

```
violations_df <- read.csv("https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv")
```

B. Your dataframe, **violations_df**, contains a list of code violations recorded repeatedly over time (2017-2024). Identify the variable that captures the date a violation has occurred and inspect it. What is the level of detail captured in this variable (weekly, monthly, etc.)? Answer in a comment.

```
head(violations_df)
```

```
##          X       Y violation_id number_violations             complaint_address
## 1 -76.1499 43.0473   2023-19961                 1 350 Montgomery St & Jefferson
## 2 -76.1499 43.0473   2023-07624                 1 350 Montgomery St & Jefferson
## 3 -76.1499 43.0473   2023-07623                 1 350 Montgomery St & Jefferson
## 4 -76.1499 43.0473   2023-07622                 1 350 Montgomery St & Jefferson
## 5 -76.1499 43.0473        84997                 1 350 Montgomery St & Jefferson
## 6 -76.1499 43.0473        84998                 1 350 Montgomery St & Jefferson
##   complaint_zip zip_median_income          SBL
## 1         13202             26271 102.-01-07.0
## 2         13202             26271 102.-01-07.0
## 3         13202             26271 102.-01-07.0
## 4         13202             26271 102.-01-07.0
## 5         13202             26271 102.-01-07.0
## 6         13202             26271 102.-01-07.0
##                                                          violation
## 1                                 2020 FCNYS- - 105.7 - Violations
## 2            SPCC - Section 27-116 (E) - Vacant Property Registry
```

```
## 3 2020 PMCNYS - Section 304.13 - Window, skylight, and door frames
## 4            2020 PMCNYS - Section 304.3 - Premises Identification
## 5                                     2015 IFC - 1031.1 - General
## 6                                 2015 IFC - 1031.2 - Reliability
##               violation_date         comply_by_date status_type_name
## 1 2023/11/28 12:33:12.093+00 2023/12/13 12:33:12+00           Closed
## 2 2023/04/21 15:16:21.407+00 2023/05/09 15:16:21+00             Open
## 3 2023/04/21 15:14:36.107+00 2023/05/09 15:14:36+00             Open
## 4 2023/04/21 15:10:38.397+00 2023/05/09 15:10:38+00             Open
## 5 2018/12/31 10:05:13.410+00 2019/01/31 10:05:13+00           Closed
## 6 2018/12/31 10:05:13.410+00 2019/01/31 10:05:13+00           Closed
##   complaint_number complaint_type_name                  open_date
## 1       2023-08038          Fire Alarm 2023/11/28 12:31:52.007+00
## 2       V2023-0567        Vacant House 2023/04/21 15:08:03.057+00
## 3       V2023-0567        Vacant House 2023/04/21 15:08:03.057+00
## 4       V2023-0567        Vacant House 2023/04/21 15:08:03.057+00
## 5       2018-36521   Unsafe Conditions 2018/12/31 10:01:30.220+00
## 6       2018-36521   Unsafe Conditions 2018/12/31 10:01:30.220+00
##                     owner_name inspector_id Neighborhood         Vacant
## 1 Shalimar Properties 407 LLC          646     Downtown Yes_Commercial
## 2 Shalimar Properties 407 LLC          908     Downtown Yes_Commercial
## 3 Shalimar Properties 407 LLC          908     Downtown Yes_Commercial
## 4 Shalimar Properties 407 LLC          908     Downtown Yes_Commercial
## 5 Shalimar Properties 407 LLC          646     Downtown Yes_Commercial
## 6 Shalimar Properties 407 LLC          646     Downtown Yes_Commercial
##      owner_address owner_city owner_state owner_zip_code Latitude Longitude
## 1 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
## 2 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
## 3 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
## 4 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
## 5 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
## 6 407 S Warren St    Syracuse          NY          13202  43.0473  -76.1499
##   ObjectId
## 1    66357
## 2    66359
## 3    66362
## 4    66469
## 5    66471
## 6    66473
```

```r
# the variable is called 'violation_date' and it records the exact date with
# the time
```

C. What is the data type of this variable? Explain in a comment and show the command you used to find out.

```r
str(violations_df$violation_date)
```

```
##  chr [1:132250] "2023/11/28 12:33:12.093+00" "2023/04/21 15:16:21.407+00" ...
```

```r
# it is a character data type and I used str() to find out the type
```

D. To properly display the time values in our plots as dates, we must convert the time variable to date format using the **as.Date()** function. Run the code below, comment on how it works, and double-check the variable's data type to ensure it is no longer coded as text.

```r
violations_df$violation_date <- as.Date(violations_df$violation_date)
# Comment: since we selected the violation_date column in violations_df and
# then used the as.Date() function on the same column, it changes the format in
# the data frame. I then used str() to confirm that it was changed
```

```r
str(violations_df$violation_date)
```

```
##  Date[1:132250], format: "2023-11-28" "2023-04-21" "2023-04-21" "2023-04-21" "2018-12-31" ...
```

Step 2: Dealing with missing data and subsetting

A. When you first start working with a dataset, it is always a good idea to look for missing values. Check the variables **number_violations** and **zip_median_income** for missing values using the **is.na()** function. Are there any?

```r
sum(is.na(violations_df$number_violations))
```

```
## [1] 0
```

```r
sum(is.na(violations_df$zip_median_income))
```

```
## [1] 22
```

```r
# there are no missing values in the number_violations column, but there are 22
# in zip_median_income
```

B. **imputeTS** is a R package that is specifically designed to 'repair' missing values in time series data. We'll use this instead of the simpler **mean substitution** method because it's more accurate. If there are missing values in the two variables, use the na_interpolation() function from the imputeTS package to interpolate the missing values. Make sure to first sort the dataframe by violation_date and then by complaint_zip before applying the function.

```r
# install.packages('imputeTS')
library(imputeTS)
violations_df <- violations_df[order(violations_df$violation_date, violations_df$complaint_zip),
    ]
violations_df$number_violations <- na_interpolation(violations_df$number_violations)
violations_df$zip_median_income <- na_interpolation(violations_df$zip_median_income)
```

C. To ensure that there is no more missing data, run the code from Step 2A again:

```r
sum(is.na(violations_df$number_violations))
```

```
## [1] 0
```

```r
sum(is.na(violations_df$zip_median_income))
```

```
## [1] 0
```

D. Create a new dataframe containing only the data for the "Northside" neighborhood using subsetting.

Hint: To avoid including rows with missing values, add this condition to your subsetting code: & !is.na(violations_df$Neighborhood). For example, filtering for the "Southside" neighborhood would look like this: [violations_df$Neighborhood == "Southside" & !is.na(violations_df$Neighborhood),]

```r
northside_df <- violations_df[violations_df$Neighborhood == "Northside" & !is.na(violations_df$Neighborh
    ]
```

E. Using the same logic, create a separate dataframe for the following neighborhood: Westcott

```
westcott_df <- violations_df[violations_df$Neighborhood == "Westcott" & !is.na(violations_df$Neighborho
    ]
```

F. Using the same logic, create a separate dataframe for the following neighborhood: Downtown

```
downtown_df <- violations_df[violations_df$Neighborhood == "Downtown" & !is.na(violations_df$Neighborho
    ]
```

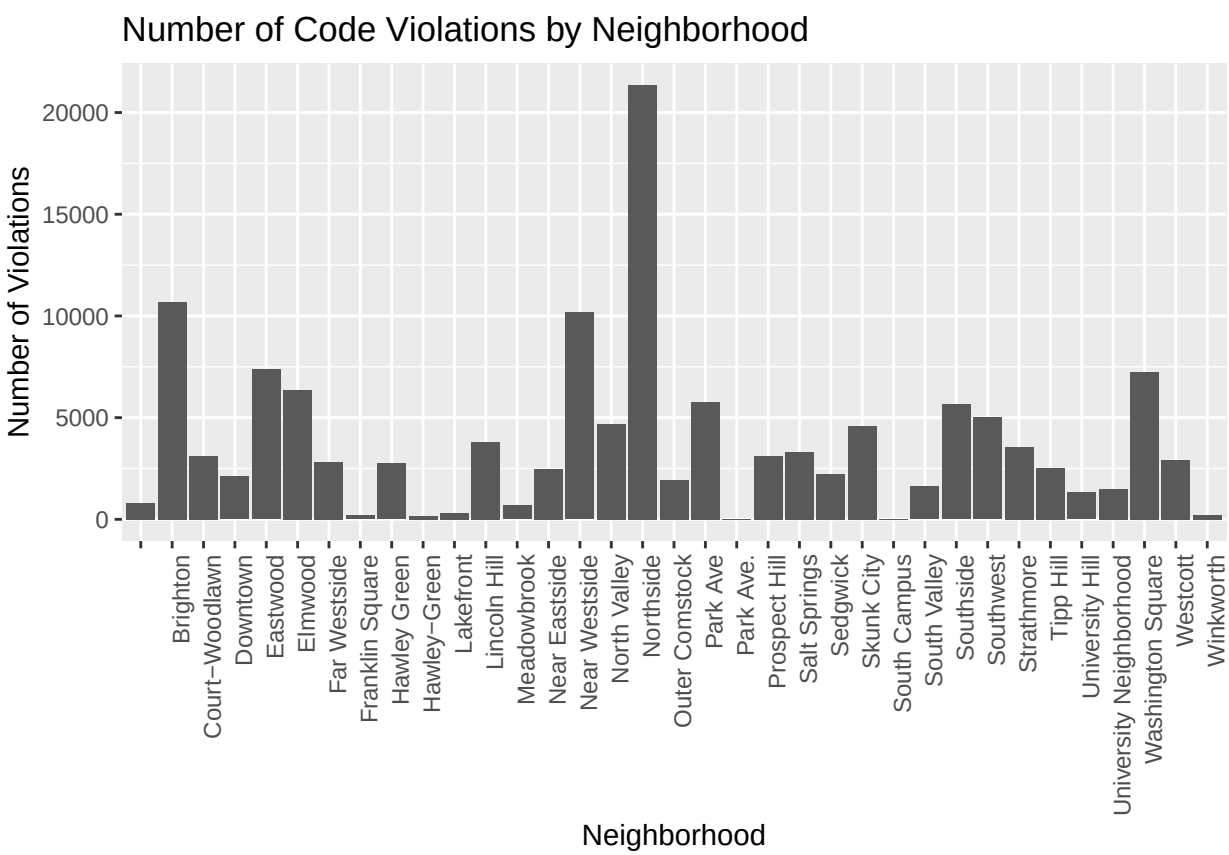
G. Finally, using the same logic, create a separate dataframe for the University area which combines two neighborhoods: University Neighborhood and University Hill.

```
university_df <- violations_df[violations_df$Neighborhood == "University Neighborhood" |
    violations_df$Neighborhood == "University Hill" & !is.na(violations_df$Neighborhood),
    ]
```

Step 3: Use ggplot to explore the distribution of the data

Don t forget to install and library the ggplot2 package. Then: A. Create a bar chart for **Neighborhood** in the **violations df** dataframe. Make sure to add a title. In a comment, briefly interpret the histogram.

```
library(ggplot2)
ggplot(violations_df, aes(x = Neighborhood)) + geom_bar() + xlab("Neighborhood") +
    ylab("Number of Violations") + ggtitle("Number of Code Violations by Neighborhood") +
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```




```
# the bar chart shows the number of violations per neighborhood, the north side
# has the highest
```

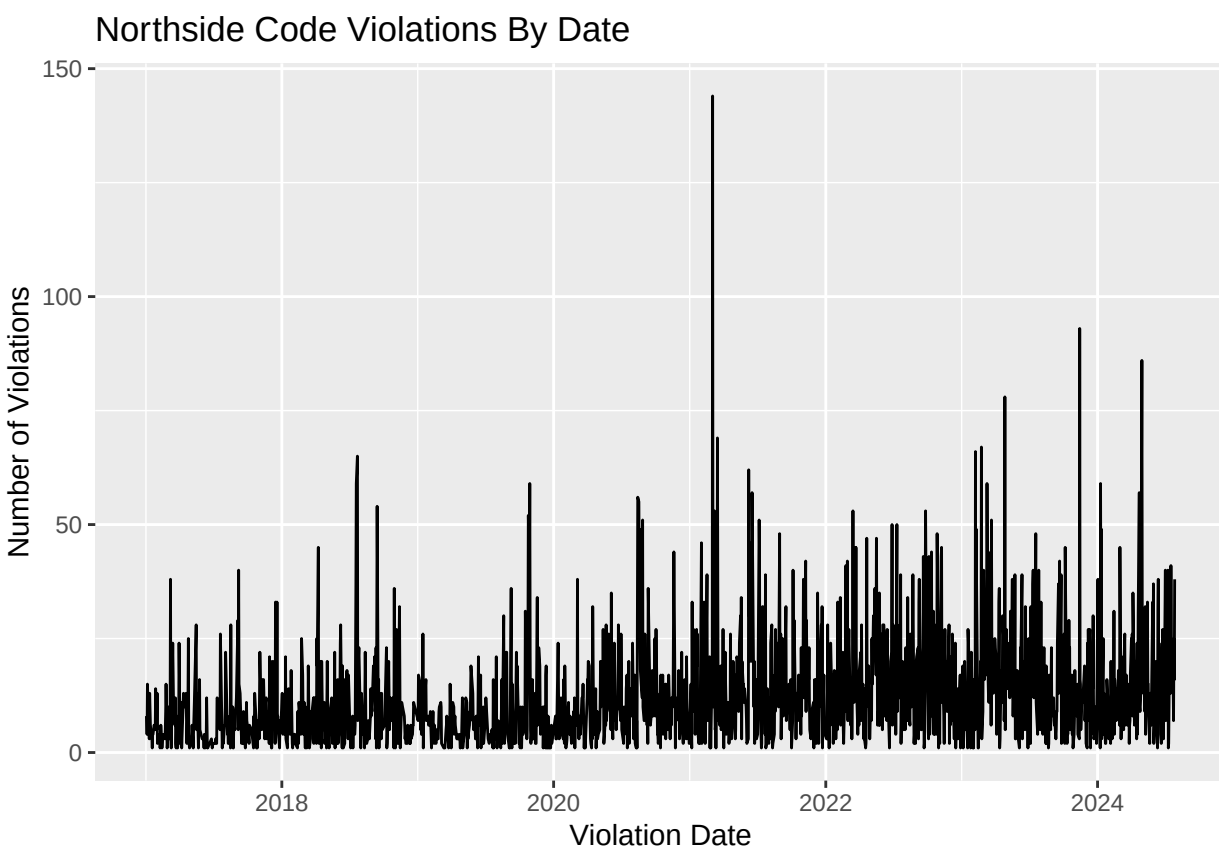
Step 4: Explore how the data changes over time

A. The data were collected over nearly 7 years. A line chart can be used to track changes over time. First, however, we need to aggregate the number of violations by date using the aggregate() function. Here is an example of how it works, applied to the overall data stored in violations_df. Use the same logic to obtain date aggregates for the four neighborhood dataframes:

```
violations_by_date <- aggregate(number_violations ~ violation_date, data = violations_df,
    sum)
downtown_violations_by_date <- aggregate(number_violations ~ violation_date, data = downtown_df,
    sum)
northside_violations_by_date <- aggregate(number_violations ~ violation_date, data = northside_df,
    sum)
university_violations_by_date <- aggregate(number_violations ~ violation_date, data = university_df,
    sum)
westcott_violations_by_date <- aggregate(number_violations ~ violation_date, data = westcott_df,
    sum)
```

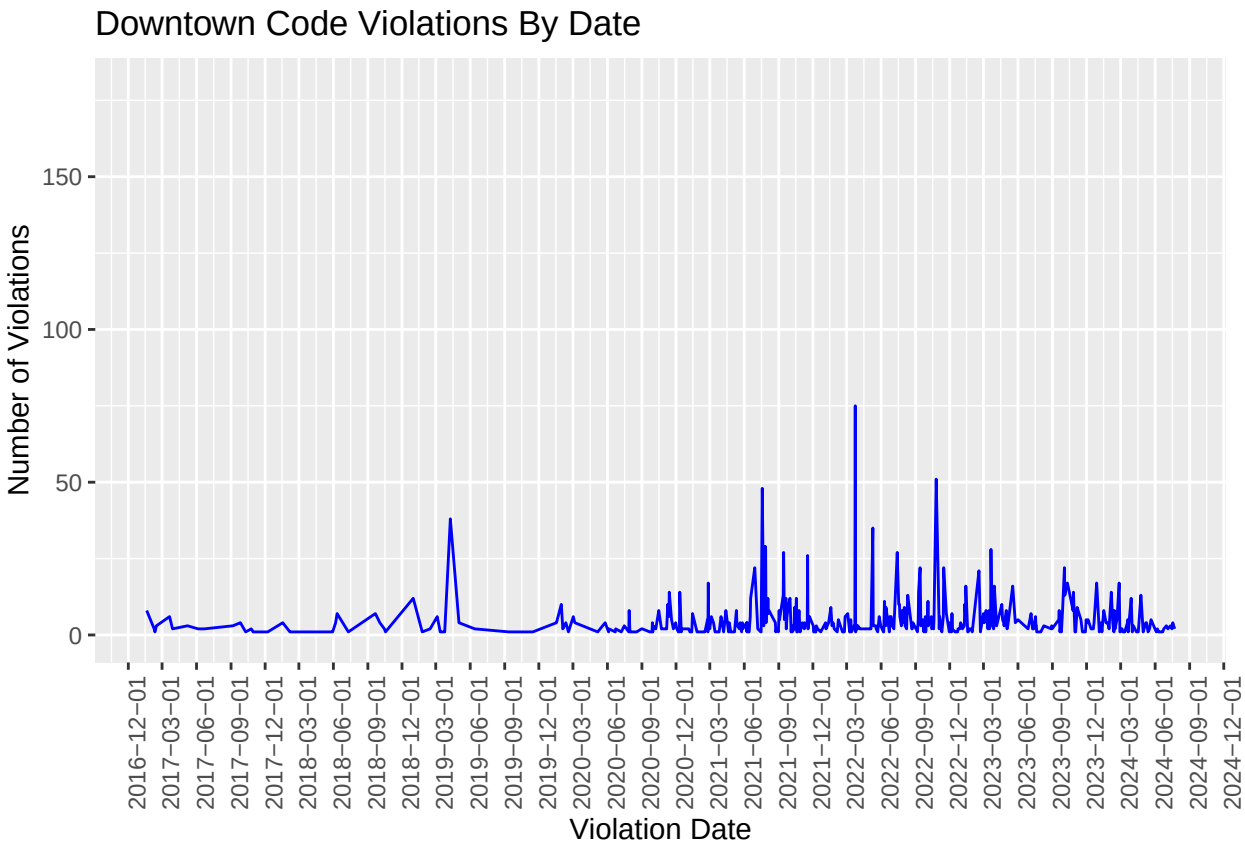
Once you are done with the previous step, we can start creating line charts to visualize the pattern of violations in each neighborhood over time. Let's focus on the **Northside df** subset first: Set the violation date variable on the X-axis and number_violations on the Y-axis of a **line chart**.

```
ggplot(northside_violations_by_date, aes(x = violation_date, y = number_violations)) +
    geom_line() + xlab("Violation Date") + ylab("Number of Violations") + ggtitle("Northside Code Viola
```

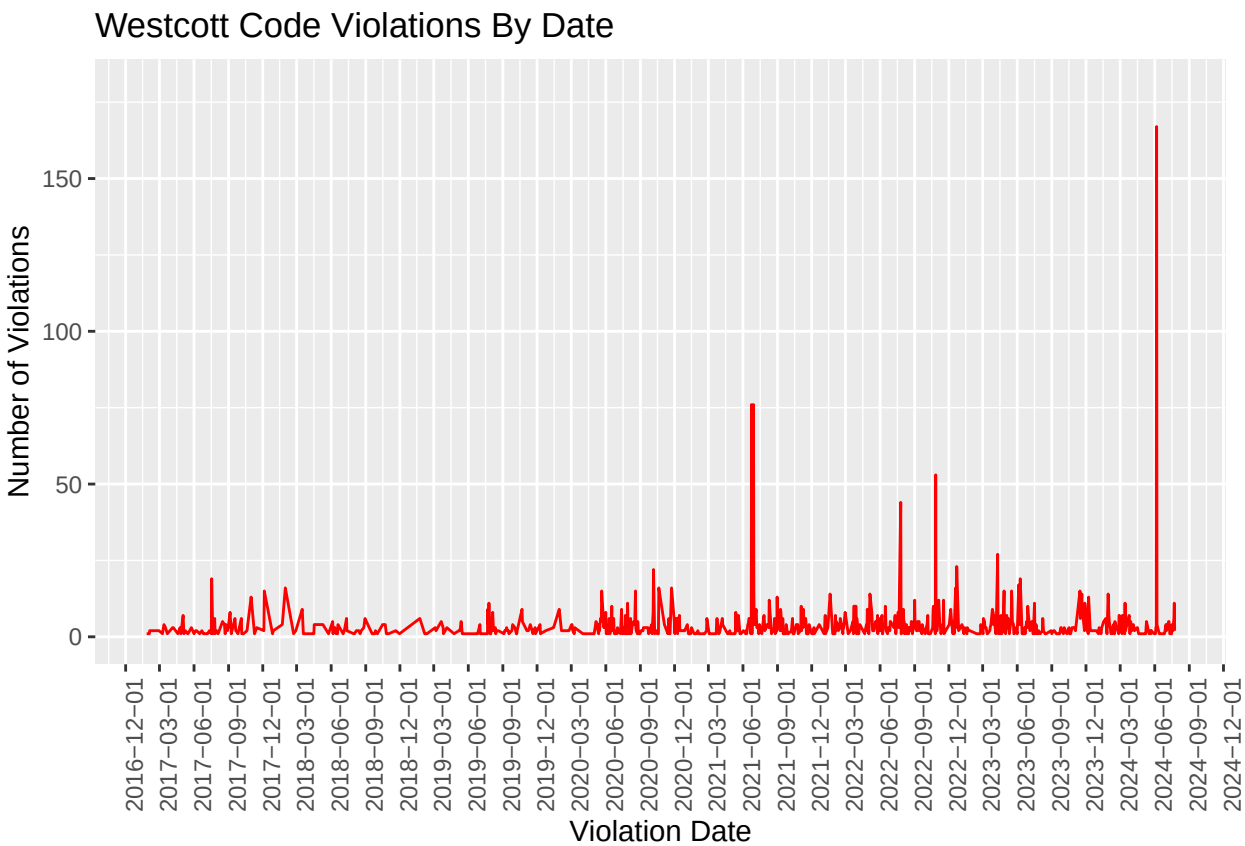


B. Next, generate similar graphs for each of the remaining three subsets. Use a different color for each line plot. For full credit, ensure that the x-axis is displayed in a more granular format, such as quarterly intervals, and set the y-axis to extend up to at least 180, e.g. by using the ylim() function. This will allow for effective comparison of patterns across the different plots.

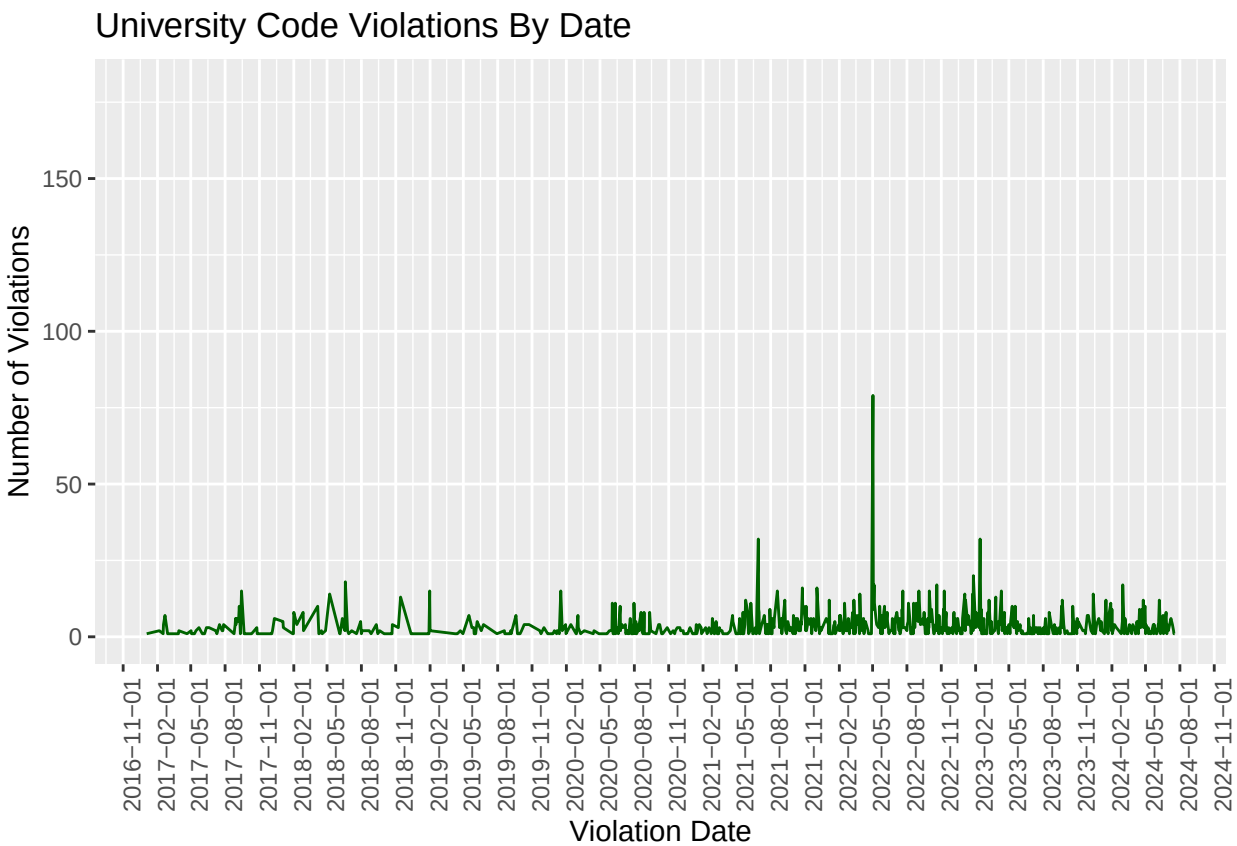
```r
ggplot(downtown_violations_by_date, aes(x = violation_date, y = number_violations)) +
    geom_line(color = "blue") + xlab("Violation Date") + ylab("Number of Violations") +
    ggtitle("Downtown Code Violations By Date") + ylim(0, 180) + scale_x_date(date_breaks = "3 months")
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```




```r
ggplot(westcott_violations_by_date, aes(x = violation_date, y = number_violations)) +
    geom_line(color = "red") + xlab("Violation Date") + ylab("Number of Violations") +
    ggtitle("Westcott Code Violations By Date") + ylim(0, 180) + scale_x_date(date_breaks = "3 months")
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```




```
ggplot(university_violations_by_date, aes(x = violation_date, y = number_violations)) +
    geom_line(color = "darkgreen") + xlab("Violation Date") + ylab("Number of Violations") +
    ggtitle("University Code Violations By Date") + ylim(0, 180) + scale_x_date(date_breaks = "3 months
    theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



C. Now, let's create a scatter plot using violations_df to explore the relationship between number_violations and zip_median_income. This code calculates the total number of violations for each zip code and visualizes the relationship. Run the code and add comments to explain what each line does.

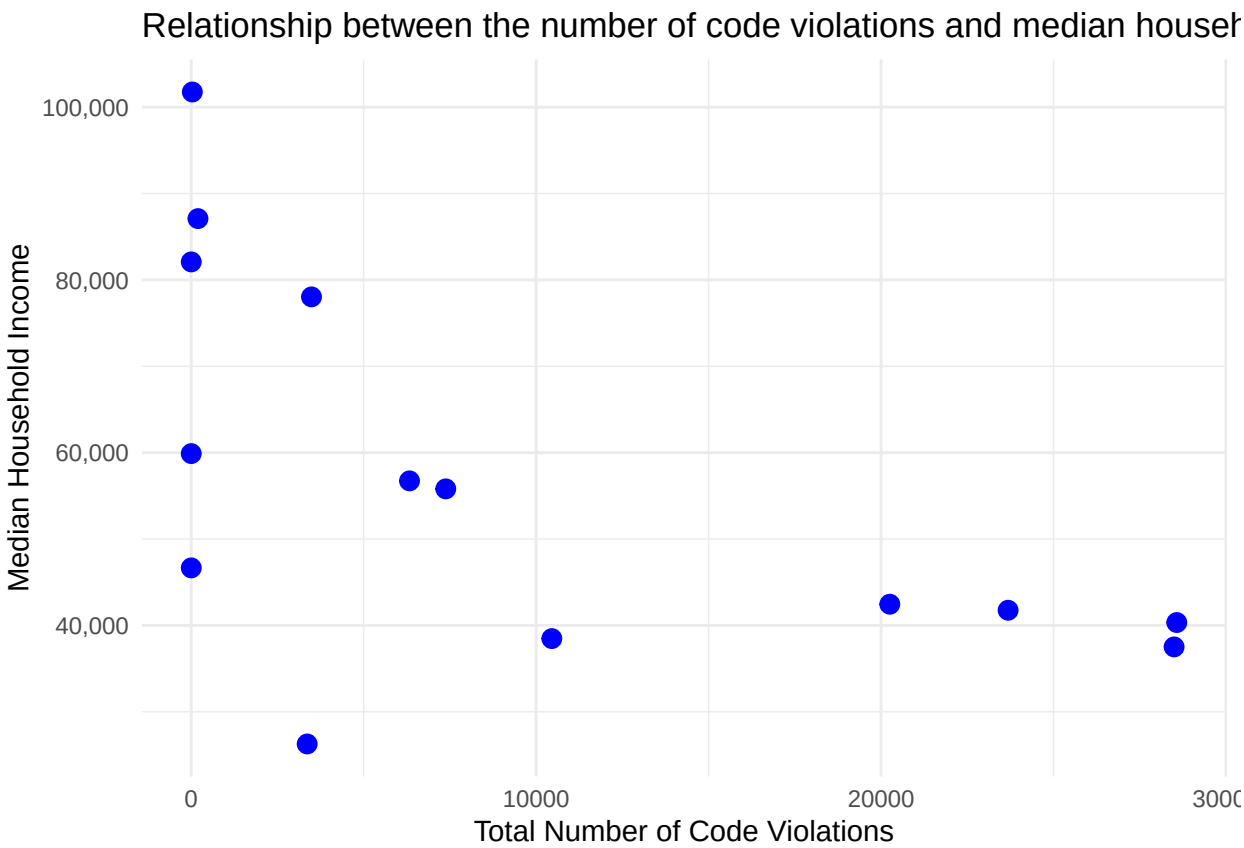
```
agg_data <- aggregate(violations_df$number_violations,
                      by = list(complaint_zip = violations_df$complaint_zip,
                                zip_median_income = violations_df$zip_median_income),
                      FUN = sum)
# aggregate the total number of violations for each zipcode and the median income for the zip code

colnames(agg_data) <- c("complaint_zip", "median_income", "total_violations")
# renames column names

ggplot(agg_data, aes(x = total_violations, y = median_income)) +
# creates scatter plot of the relationship between total violations and median income

  geom_point(color = "blue", size = 3) +
  #plot each relationship as a blue dot

  ggtitle("Relationship between the number of code violations and median household income in Syracuse, N
  xlab("Total Number of Code Violations") + #add x-axis label
  ylab("Median Household Income") + # add y-axis label
  scale_y_continuous(labels = function(x) format(x, big.mark = ",", scientific = FALSE)) + # format the
  theme_minimal() # apply a minimal theme
```



Step 5: Executive summary

A. In 3-4 sentences, summarize the results from steps 3 and 4 - based on your interpretation of the data, do you see any patterns within and between the line charts and scatterplot? What story do these charts tell?

```
# I noticed that all of the line plots showed that the neighborhoods code
# violations vary over time, they mostly stay around the same range but all of
# them had a spike in the amount of violations at differing times. There was
# not a correlation between the dates of the spikes which means there wasn't a
# universal event that caused them. The bar plot showed us that there were two
# areas with zero violations, and the north side had a substantially larger
# amount of violations. The scatter plot showed us that there was a correlation
# between the median income and code violations, most times the higher the
# income the less violations the area had.
```